Trabalho Prático - Grupo11 – Dados e Aprendizagem Automática

Augusto Campos, Carlos Silva, Tiago Silva, and Tiago P. Silva

[1] Universidade do Minho, 4710-057 Braga, Portugal
[2] Dados e Aprendizagem Automática
[3] {pg57510, pg57518, pg57614, pg57617}@alunos.uminho.pt

1 Introdução

Este relatório é desenvolvido no âmbito das atividades práticas da Unidade Curricular de Dados e Aprendizagem Automática, com o objetivo de aprofundar os conhecimentos adquiridos sobre Machine Learning ao longo do semestre, através da execução de tarefas propostas pela equipa docente.

Na primeira tarefa, denominada *Analysis and Validation Task*, realizamos a exploração, análise e preparação dos datasets fornecidos (DShippo e DSocc), com o objetivo de extrair conhecimento relevante no contexto do problema de diagnóstico de demência. Além disso, utilizamos o dataset de controlo (DSocc) e os modelos desenvolvidos na segunda tarefa para calcular métricas como F1-score, entre outras, de forma a verificar a relevância do hipocampo na previsão da evolução de demências e a irrelevância do lobo ocipital.

A segunda tarefa, denominada *Competition Task*, consiste no desenvolvimento de modelos de Machine Learning otimizados para o dataset de competição (DShippo), disponibilizado através da plataforma Kaggle. Para tal, formámos equipas e seguimos as orientações da competição, incluindo a submissão diária de resultados no Kaggle para avaliação da métrica F1 macro. Durante este processo, foram realizados testes e ajustes contínuos, com o objetivo de alcançar os melhores resultados possíveis.

2 Tarefa: Análise e Validação

Os dois *datasets* fornecidos, *DShippo* e *DSocc*, representam a extração de características radiónicas de diferentes regiões do cérebro: o hipocampo e o lobo occipital, respectivamente. O *dataset DShippo* foi selecionado devido à sua relevância significativa na pesquisa sobre a doença de Alzheimer, visto que o hipocampo está amplamente relacionado com a memória e funções cognitivas, que são frequentemente afetadas na progressão da doença de Alzheimer. Por outro lado, o *dataset DSocc* foi escolhido como um controle, uma vez que o lobo occipital não está tipicamente associado ao desenvolvimento de demência, oferecendo um contexto para comparar e contrastar com a região de interesse.

No contexto desta análise, a hipótese é que as características radiónicas extraídas do hipocampo podem revelar diferenças significativas entre pacientes

com ***Mild Cognitive Impairment*** **(MCI)** que evoluirão para a **doença de Alzheimer (AD)** e aqueles que não evoluirão. Esses padrões nas características radiónicas poderiam, portanto, contribuir para a construção de modelos preditivos capazes de identificar pacientes em risco de progressão para Alzheimer.

Apesar da importância dos dados, os conjuntos apresentam uma característica desafiadora: o número de entradas é relativamente pequeno (305), enquanto o número de características (*features*) extraídas de cada amostra é extremamente elevado (2181). Essa alta dimensionalidade em relação ao número de observações dificulta uma análise profunda e direta, uma vez que modelos tradicionais podem sofrer de problemas como o overfitting e baixa capacidade de generalização. De maneira a testar diferentes conjuntos de *features* usamos diversos modelos/algoritmos como *XGBoost, Lasso, RFE, SHAP*. Além disso, removemos também as features não númericas que não apresentavam informação relevante para o problema.

Para testar a hipótese proposta, que visa verificar se o hipocampo é, de fato, relevante na previsão da evolução para Alzheimer, em contraste com o lobo occipital, foram avaliados quatro modelos diferentes em ambos os conjuntos de dados. Os resultados obtidos para cada modelo foram os seguintes:

2.1 Resultados

Dataset	Logistic Regression		Decision Tree		Lasso		XGBoost	
	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy
DSooc	0.3074	0.3115	0.1992	0.2131	0.1054	0.1311	0.3473	0.3607
DShippo	0.4395	0.4426	0.3833	0.3770	0.3640	0.3607	0.4043	0.4262

Tabela 1. Resultados de diversos modelos em relação aos 2 datasets

Logistic Regression com MinMaxScaler: Hiperparâmetros: solver=newton-cg, random_state=2021, max_iter=5000.

Decision Tree: Hiperparâmetros: max_depth=6, random_state=2021 (Os hiperparâmetros foram escolhidos após realizar um gridSearch.)

Lasso: Hiperparâmetros: alpha=0.01, random_state=2021 (Os hiperparâmetros foram escolhidos após realizar um gridSearch.)
Este modelo foi usado pela sua habilidade de regularização que também ajuda na seleção de variáveis, reduzindo o risco de *overfitting* ao forçar algumas *features* a terem coeficientes nulos, destacando assim as variáveis mais relevantes para o modelo.

XGBoost: Hiperparâmetros: colsample_bytree= 0.8, learning_rate= 0.1, max_depth= 5, n_estimators= 200, subsample= 0.8, random_state= 2021 (Os hiperparâmetros foram escolhidos após realizar um gridSearch.)
Este modelo foi usado devido à sua capacidade de selecionar as *features* mais importantes, permitindo identificar quais variáveis têm maior impacto na previsão do desenvolvimento de Alzheimer.

Através destes testes, conseguimos observar uma diferença significativa entre os resultados obtidos nos dois *datasets*, o que nos permite concluir de forma robusta que o hipocampo desempenha um papel fundamental na previsão da evolução do Alzheimer. Esta discrepância sugere que a atividade ou alterações no hipocampo têm uma influência determinante no desenvolvimento da doença, reforçando sua relevância no contexto das avaliações clínicas e prognósticas.

3 Tarefa: Competição

3.1 Tratamento dos dados

Dados não numéricos Uma das primeiras coisas que detetamos foi a presença de colunas não numéricas. Após análise, chegou-se à conclusão que não tinham relevância por isso não houve tentativas de fazer *Feature Engineering*, isto é, gerar novos atributos com a informação dessas colunas. Assim sendo, essas colunas foram simplesmente removidas.

Feature selection

Correlação Retirar uma das features de um par com mais de 80% de correlação foi uma delas, com o intuito de remover colunas com informação redundante, visto que, quando duas colunas estão altamente correlacionadas, apenas uma delas é necessária para extrair informação dos dados. O *threshold* utilizado foi de 0.8. Após esta abordagem, o dataset ficou com 529 colunas. No entanto, esta não foi a nossa principal abordagem no que toca a *Feature selection*.

Variância e Variância Retirar features com uma variância inferior a 1%, que não trazem informação útil para o problema, depois desta alteração o dataset ficou com 1128 features. De seguida, aplicou removeu se features que tivessem correlação acima de 95%, ficando com um total de 580 features.

LASSO Como dito anteriormente, LASSO força certas features a terem coeficientes nulos, dando mais relevancia a features com mais importancia, ficando apenas com 71 features.

RFE Também foi usado Recursive Feature Elimination de maneira a escolher um número ou percentagem das features mais importantes.

PCA No contexto de modelos como o *Bagging*, *Random Forest* e *Gradient Boosting*, também exploramos reduzir a dimensionalidade com PCA (*Principal Component Analysis*). Foi implementado com a ajuda de *Pipeline* que facilitou a aplicação das mesmas transformações aos datasets de treino e teste e permitiu a sua integração com uma *Gridsearch* que englobava não só os hiperparâmetros do modelo, mas também diferentes valores de *pca_ n_ components*.

SHAP Foi usado para no contexto de explicar no sentido de saber as features com mais impacto num modelo de *Decision Tree*.

3.2 Modelos desenvolvidos

Overview Foram explorados diversos algoritmos de previsão entre eles:

- Decision Trees
- Logistic Regression
- ElasticNet
- Random Forest
- Support Vector
- Multilayer Perceptron
- Gradient Boosting
- Stacking
- Bagging
- XGBoosting
- KNN
- SVM

Neste relatório vamos explorar mais em detalhe os algoritmos ***Multilayer Perceptron***, O desenvolvimento dos vários modelos foi acompanhado com submissões diárias na plataforma Kaggle, mas também se procedeu a treinar algoritmos com uma fração dos dados para testar localmente. Os testes feitos localmente foram feitos com um testsize de 0.2 e foi usado random_state=2021 para garantir a reprodutibilidade.

Multilayer Perceptron Sendo este tipo de modelo beneficiário de normalização, procedemos à normalização das dados para intervalos de 0 a 1. Não foi feita redução de dimensionalidade, porque este modelo é eficiente a ignorar ruído. Usamos o MLP_Classifier presente na biblioteca *sklearn.neural_network*. Este modelo permite a construção de modelos de redes neuronais com diferente números de camadas e neurónios. Fizemos uso de *Gridsearch* para afinar os hiperparâmetros de forma comedida, visto que *grids* muito complexas demorariam imenso tempo a correr (principalmente modleos com *learning rates* muito pequenos). Estes foram os parâmetros que usamos.

- *hidden_layer_sizes*: (50,25,25)
- *activation*: *logistic*
- *alpha*: 0.1
- max_iter: 5000

Localmente obtivemos 0.5082 de *accuracy* e 0.4890 de *F1 Score*. Infelizmente, o score privado no Kaggle foi de 0.33.

Logistic Regression Este é também um modelo que beneficia de normalização como o *Multilayer Perceptron*. Reduzimos a dimensionalidade com base no método de correlação elevado de 0.8 entre colunas. Neste exemplo não explorámos os hiperparâmetros. O solver usado foi o *newton-cg*. Localmente obtivemos 0.4262 de *accuracy* e 0.4269 de *F1 Score*. O score privado no Kaggle foi de 0.35813.

XGBoosting Este modelo não necessita intrinsecamente de Feature Selection, pois lidam bem com grandes numeros de features, porém a implementação do mesmo levou a uma melhoria significativa nos resultados na competição. Aplicamos 2 métodos de Feature Selection utilizando o *RandomizedSearch* para obter os hiperparâmetros:

LASSO com um score privado no Kaggle de 0.37404.

- *colsample_bytree:* 0.9847923138822793
- *gamma:* 3.8756641168055728
- *learning_rate:* 0.29184968246925674,
- *max_depth:* 6,
- *n_estimators*: 369,
- *subsample:* 0.8636359979282104

Variância e Correlação com um score privado no Kaggle de 0.37798.

- *colsample_bytree:*0.6792328642721364
- *gamma:* 0.5793452976256486
- *learning_rate:* 0.2689310277626781
- *max_depth:* 6
- *n_estimators*: 879,
- *subsample:* 0.6654490124263246

KNN *K-nearest neighbors* é um algoritmo que beneficia bastante da redução de dimensionalidade que foi feita durante a Feature Selection. Este algoritmo junto com a Feature Selection de Variância e Correlação obteve o nosso melhor score no Kaggle com um score de 0.38075. Os hiperparâmetros foram selecionados através de *GridSearch*.

- *metric:* manhattan
- *n_neighbors:* 5
- *weights:* uniform

4 Análise Crítica

O fluxo de desenvolvimento deste trabalho prático não foi o ideal. Uma das principais lições que tiramos do trabalho é que no futuro devemos basear as nossos avanços no desenvolvimento dos modelos em boas práticas de validação. A equipa guiou-se erradamente pelos scores públicos da plataforma *Kaggle*, que era uma amostra muito pouco significativa da qualidade do modelo submetido.

Esta abordagem teve um impacto especialmente negativo no que toca a *scores* enganadores quando submetíamos modelos em que os dados haviam sido submetidos a processos de *Feature Selection* e observávamos piores resultados. Isto levou, por vezes, a não seguirmos a direção correta de desenvolvimento de um bom modelo, principalmente no que toca a explorar os limites de um determinado modelo. Por isso é que é visível uma exploração muito vasta no que toca a número de diferentes algoritmos, embora seja questionável em alguns deles a profundidade de exploração, no que à redução de dimensionalidade diz respeito.

Outro ponto que merece atenção, foi a dificuldade na exploração do dataset de 305 linhas e 2181 colunas. Difícil de encontrar padrões e ligações entre os dados com ferramentas visuais, tal como foi feito nas aulas práticas. Ainda assim, como já foi mencionado noutro capítulo, encontramos alternativas para fazer redução de dimensionalidade, embora não tendo a oportunidade de analisar as variadas colunas manualmente. A forma do dataset fez dificultou e tornou lenta a interpretação do mesmo, limitando abordagens criativas como a de separar a coluna objetivo em duas, de modo a prever com dois modelos e dois conjuntos de atributos o objetivo original. Esta tarefa é impraticável sem o total domínio das colunas do dataset em relação ao que cada uma delas representa (se são do exame do mês 0 ou do mês 24, por exemplo).

O número de linhas do dataset também foi um factor limitador e isso é facilmente observável não só pelos resultados da equipa, como também pelos resultados de outros colegas observados na *Leaderboard* da competição no Kaggle. Esses resultados sugerem que os modelos estão *overfitted* e com baixa capacidade de generalização.

Outro aspeto sobre o qual nós não estamos satisfeitos é a organização geral dos nossos *notebooks*. Poderia ter sido mais explorada a componente *Markdown* dos próprios, por exemplo.

Em conclusão, podemos observar pela tabela 2 que foi através de métodos de *Feature Selection* e de certo tipo de modelos como o KNN e modelos *Ensemble* (XGBoost, Bagging,...) que conseguimos obter melhores resultados na competição. No entanto, na classificação final o modelo escolhido foi erradamente aquele com melhor score público e essa foi mais uma consequência do primeiro ponto deste capítulo. Após toda esta dinâmica, sentimo-nos bem mais preparados para desenvolver outros projetos deste ramo da Engenharia Informática onde tentaremos evitar os mesmos erros que é o objetivo mais importante enquanto estudante e potencial futuro Engenheiro de Dados.

Submissão	**Private Score**	**Public Score**
KNN_after_FS.csv	0.38075	0.24410
XGBoostafterFS.csv	0.37798	0.36339
XGBoost_after_Lasso.csv	0.37404	0.30666
KNN.csv	0.35976	0.28818
LRegression_80corr.csv	0.35813	0.26106
Bagging_LR_PCA.csv	0.35666	0.29269
LR_solverSaga2.csv	0.34798	0.30333
SVM_v1.csv	0.34594	0.26464
LRegression_StandardScaler.csv	0.34587	0.29833
DT_n160.csv	0.34157	0.35078
Bagging_DT_PCA.csv	0.34119	0.27541
LR_solverSaga.csv	0.33885	0.42344
LR_gridsearch.csv	0.33183	0.41868
MLPClasifier_local.csv	0.33030	0.25505
XGBoostafterFS2.csv	0.32800	0.37777
svm_c100.csv	0.32665	0.28939
XG_Boost.csv	0.32607	0.28444
SVMafterFS.csv	0.32542	0.27076
LR_RFE_top100features.csv	0.32232	0.31807
KNN_after_Lasso.csv	0.32147	0.38216
lr_top500.csv	0.31604	0.35565
RT_after_Lasso.csv	0.31442	0.24434
XGBoostafterFS2.csv	0.31096	0.39230
LR_top1200.csv	0.30437	0.31200
MLPClasifier_v1.csv	0.30410	0.41838
stackingClassifier.csv	0.29111	0.29211
svm_sem_CN-MCI.csv	0.28407	0.23021
LRegression_MinMaxScaler.csv	0.28365	0.45670
KNNafterFS2.csv	0.28309	0.26095
Log_top10.csv	0.28195	0.42945
SVM_after_Lasso.csv	0.28115	0.28111
DT_max6_top10.csv	0.27830	0.39538
DT_max6_SMOTE.csv	0.27753	0.28567
LR_after_Lasso.csv	0.27543	0.26523
decision_tree_predicts_maxdepth4.csv	0.27522	0.35085
decision_tree_predicts_maxdepth5.csv	0.27116	0.39859
DT_age_subIntervals.csv	0.26868	0.37460
LR_SMOTE.csv	0.26572	0.45670
naiveBaysNaoNormalizado.csv	0.25963	0.35238
svm_c100_linear.csv	0.24947	0.38926
DT_top1200fromLR.csv	0.24213	0.43458
decision_tree_predicts_maxdepth6.csv	0.23899	0.43533
logistic_regression_predicts.csv	0.23860	0.31272
decision_tree_predicts_maxdepth7.csv	0.23401	0.37238
ridge_alpha100.csv	0.21316	0.36426
decision_tree_predicts.csv	0.20946	0.40180
lasso_alpha01.csv	0.18799	0.42718

Tabela 2. Scores do Kaggle